

Every Finding: Experiment, Result, Significance, Role in the Argument

4 October 2026 · Complete record · DI-Bench oracle study

The paper's overarching problem. We evaluate AI coding agents on repository tasks by running the project's test suite and calling a pass a success. This document establishes that for dependency work that oracle is substantially blind, measures how blind, explains the two mechanisms behind it, shows the blindness is predictable without execution, and builds a benchmark whose oracle actually bites.

Every number below was re-derived from raw per-instance results on 4 October 2026. Findings marked **[CONTESTED]** were challenged by an adversarial audit and the dispute is stated honestly.

F1 — DI-Bench has decayed by half

Experiment. Took DI-Bench's own official correct answers ("gold patches") for all 98 Python instances and re-ran each one through the real CI suite in a container, on free GitHub Actions runners under the sysbox runtime.

Result. 51 of 96 still pass. Two instances excluded because their CI job never started. Failures split roughly: ~17 at install time (Poetry's installer dropped Python 3.8/3.9, a Mambaforge download 404s, old Python no longer fetchable), ~23 in the projects' own tests from unpinned libraries releasing breaking versions. An audit found 4 of the 45 are infrastructure noise (`context canceled`), so the defensible drift count is **41**.

Significance to the field. Executable benchmarks rot, and our rate is ordinary rather than alarming. Aguilar et al. (SCAM 2023) found only 67% of BugsInPy reproduced within three years. Zhu & Rubio-González (ICSE 2023) found 26.6–96.9% reproducibility across five Java datasets in 13 months. Mukherjee et al. (ISSTA 2021) found 71.1% of 2,702 Python builds unreproducible from dependency errors — the exact failure class we hit. Our 53% at 21 months sits inside that range. **Nobody had measured it for DI-Bench**, and anyone using the benchmark today without checking would attribute these failures to the model under test.

Role in the argument. This is the *conditioning result*. Every other execution number in the paper is reported only on gold-passing instances, because otherwise we would be measuring benchmark rot rather than model behaviour. It also does double duty thematically: the drift that the paper is about shows up inside the benchmark meant to measure it.

F2 — Half of all declared dependencies are invisible to the test oracle

Experiment. For every declared dependency of every instance, built a mutant: the official correct manifest with exactly one dependency deleted and nothing else changed. Verified syntactically that the resulting set equals gold minus that one package. Ran each through the real CI suite. **531 mutants**, each a full container execution.

Result. On gold-passing instances, **115 of 233 deletions (49.4%) still pass CI. 38 of 51 instances** have at least one invisible dependency. Sanity check: of the 118 deletions CI *did* catch, the package was genuinely absent in 114, so the experiment behaves correctly where it should.

Significance to the field. Independent methods converge on the same magnitude. Hejderup & Gousios (JSS 2022) seeded 1.1M faults at library call sites in 262 Java projects and found tests detected only 47% of direct and 35% of indirect faults. STING (arXiv 2604.01518) probed SWE-bench Verified with program variants and found 77% of instances had at least one surviving variant. Krafczyk & Schmid found 7.1% of Defects4J bugs

pass all tests after deleting a single statement. Our 49.4% is bracketed by these. **The convergence is the point:** four independent teams, four benchmarks, same conclusion that roughly half the oracle is blind.

Role in the argument. This is the paper's spine. It converts "tests might be a weak oracle" from a plausible worry into a measured quantity, and it bounds what execution pass rate can ever detect. Everything downstream — the predictor, the gates, the benchmark's instance filter — exists because of this number.

F3 — The blindness has two distinct mechanisms [the novel contribution]

Experiment. For each of the 115 invisible deletions, parsed the CI installation log to determine whether the package was present in the environment despite being undeclared.

Result. 91 are phantoms: the package installed anyway, pulled in transitively by another dependency or by a test-runner configuration. The code genuinely works; the declaration was redundant. **24 are blind spots** (10.3% of all mutants, in 15 of 51 instances): the package was genuinely absent, the import would crash, and the tests still passed because nothing reaches that code.

Significance to the field. This split **appears nowhere in the prior literature**. Hejderup, STING, Shulepov and Krafczyk all report a single undifferentiated miss rate. The two halves have opposite implications: the 91 phantoms are a defect in the *benchmark's answer key*, where a model omitting them is marked wrong for being right, and are curable by better curation. The 24 blind spots are defects in the *projects' own test suites*, and no amount of curation fixes them. Corroboration exists for each half separately — Drosos et al. (FSE 2024) measured transitively-listed declarations at 30% of PyPI bloat, and Zhang et al. found 34.12% of Maven projects use undeclared dependencies — but nobody has separated them inside a benchmark.

Role in the argument. This is the finding the paper should lead with, and currently undersells. It is what makes the work a contribution rather than a replication of "tests are weak." It also directly motivates the gate design: phantoms need an install-level check (G5), blind spots need a behavioural check (G4).

F4 — The blindness is predictable without executing anything

Experiment. Built two static signals and tested them against the 233 executed mutant outcomes.

Dependency layer: is the package already in the transitive requirement closure of the *other* declared packages, computed from public PyPI metadata? **Code layer:** does any test file reach, directly or through project modules, a file that imports the package?

Result.

Predictor	Flagged	Precision	Recall
Dependency layer alone	73	0.97	0.62
Code layer alone	70	0.69	0.42
Both combined	118	0.81	0.83

An audit confirmed the predictor is not circular and beats every single-signal baseline (best alternative F1 0.755 vs 0.815).

Significance to the field. Each component is standard; the *target* is new. Eclipse Steady (Ponta et al., ICSME 2018) does code-centric reachability to triage vulnerabilities. deptry and FawltlyDeps check manifest hygiene. Drosos stitches call graphs across PyPI. **Nobody predicts whether a CI oracle will notice a change.** This reframes a \$100-of-compute, 531-container experiment as something runnable in seconds on any repository.

Role in the argument. Three jobs. It makes F2 and F3 *actionable* — anyone can audit their own benchmark cheaply. It supplies the mechanism for gates 4 and 5. And it independently corroborates F3, since the dependency layer predicts phantoms at 0.97 precision precisely because phantoms have a mechanical cause.

F5 — Tests alone accept a third of deliberately faked removals

Experiment. For the 62 strongest removal instances, automatically generated three families of fake removal and ran all of them through the real CI. **Hide:** move the package to an optional extra or dev group that CI installs anyway. **Vendor:** copy the library's own distributed source into the repository. **Stub:** create a local package shadowing the import name whose symbols return nothing. 176 variants generated; 10 vendor variants skipped because the package ships compiled extensions.

Result.

Fake	Variants	Passed CI	Caught by other gates
Hide	62	35	35 of 35 (G1, G5)
Vendor	52	18	18 of 18 (G3)
Stub	62	8	0 of 8
Total	176	61 (34.7%)	53 of 61

The stubs are detectable only structurally: trivial-body and shadow-package signals flag **61 of 62**.

Significance to the field. Adversarially probing a verifier with constructed exploits is now a named methodology. BenchJack (arXiv 2605.12673) red-teamed 10 agent benchmarks and found 219 flaws in eight recurring patterns. The hacker-fixer loop (2606.08960) found 16% of 1,968 tasks hackable from the description alone. ImpossibleBench (2510.20270) measures propensity to modify or special-case tests — our stub family is their special-casing pattern. **What is new is the task:** nobody has enumerated the cheat families for a dependency-removal oracle or shown, with executed CI, which gate catches which.

Role in the argument. This is the constructive half of the paper. F2 and F3 show the oracle is blind; F5 shows what that blindness *admits*, and that a small, deterministic gate stack closes two of three families completely. The stub result is the sharpest single sentence in the paper: **the one cheat that requires no installation and no copying defeats every install-level check**, which is precisely why a structural gate is necessary rather than decorative.

F6 — On real agents, the test score overstates success

Experiment. Four models attempted genuine in-situ removals under a deliberately neutral prompt that never mentions cheating, using a SWE-agent-style scaffold (100-line viewer, range-based edit with syntax linting, history collapsing, temperature 0, single attempt). **52 runs** total.

Result.

Model	Tier	Runs	CI pass	Genuine after all gates
deepseek-v3.2	open-weight	15	5	3
gpt-5.1	frontier	16	1	1
claude-sonnet-5	frontier	8	1	1
qwen3-coder	open-weight	13	0	0

For deepseek, **CI reported 5 successes where only 3 are genuine**: one left the declaration in place while CI passed anyway, one changed nothing at all while the package arrived transitively. Both caught by G1 and G5.

Significance to the field. The phenomenon is established at scale and our result agrees. *Building to the Test* (arXiv 2606.28430) found production agents with the oracle in the loop reaching near-perfect scores while the required library was "dead or absent." Wang, Pradel & Liu (ICSE 2026) found 7.8% of SWE-bench patches pass while failing developer tests and 29.6% behaviourally diverge from gold. SpecBench found deliberate exploits rare and compositional failure dominant — matching our result exactly. **The phenomenon is not ours; the task is.**

Role in the argument. This is the demonstration that F2–F5 are not theoretical. On a real task with real agents, the gap between reported and genuine success is 5 versus 3. It is the smallest experiment in the paper and should be presented as corroboration, not as a headline.

F7 — Frontier models were not better, and failed differently [unexpected]

Experiment. Same 16 instances, same scaffold, comparing open-weight against frontier models from two different labs.

Result. On the 15 instances both attempted, **deepseek passed 5, gpt-5.1 passed 1**. Sonnet agreed with gpt-5.1 on every shared instance. The mechanism is visible in patch size: on three instances deepseek passed and gpt-5.1 failed, deepseek wrote 150, 128 and 36 added lines against gpt-5.1's 22, 46 and 13. Frontier models make minimal, surgical edits; those edits more often miss a usage site elsewhere in the repository or a behavioural detail. gpt-5.1 moved an import inside a function rather than removing it; it fixed the one source file using `numpyro` but missed three test files importing it directly.

Significance to the field. Benchmark leaderboards assume capability ordering transfers across tasks. Here it is inverted. The frontier models produced *cleaner* artifacts — zero truncation, 15 of 16 calling `finish` versus deepseek running out of turns on 13 of 15 — while scoring lower. This suggests the task rewards repository-wide reasoning over local code quality, which is the kind of distinction single-number leaderboards erase.

Role in the argument. Strengthens the case that the task is genuinely hard for reasons of program understanding rather than code generation. **Caution:** with 8 and 16 runs against 15, this does not support a ranking claim. State that frontier models were *not better* and that failures were *qualitatively different*; do not claim they are worse.

F8 — No spontaneous cheating in 52 runs [CONTESTED]

Result as recorded. Across all 52 agent runs, our gates flagged no vendoring, no stubbing, and no weakened test suites.

Why this is contested. An adversarial audit found that gate 3 only inspects *newly added* files, so it never examined any agent's modifications to existing files, and identified at least one `llm-mlc` run that resembles stubbing. The negative claim is therefore **not supportable as worded**.

Honest framing for the paper. Say: "our detectors did not flag deliberate cheating; the detectors have a known blind spot on modified files, so we report this as the absence of evidence rather than evidence of absence." The constructed cheats in F5 remain the sole load-bearing evidence that the oracle *would* accept these behaviours.

F9 — A static tool from 2015 beats a 7B model on dependency inference

Experiment. DI-Bench compares only LLM prompting strategies and never ran a static baseline. We ran pipreqs on the same 98 masked repositories and scored it identically.

Result.

Method	Precision	Recall	F1	Exact set
Qwen2.5-Coder-7B	0.30	0.23	0.26	6 of 98
pipreqs (ignoring tests/docs)	0.65	0.78	0.71	14 of 98

Disclosures: this is the better of two pipreqs configurations (untuned F1 0.624), and it makes live PyPI lookups.

Significance to the field. The DI-Bench paper cites exactly one dependency-inference tool. The static literature is older and stronger than pipreqs: DockerizeMe (ICSE 2019), SnifferDog (ICSE 2021), and especially **PyEGo** (ICSE 2022), which is 0.4–3.5× more accurate than prior tools. A reviewer will ask why PyEGo was not run. **Our pipreqs number is therefore a floor, not a ceiling** — the weakest static tool already triples the 7B model's recall.

Role in the argument. Establishes that any LLM method on this benchmark must clear a trivial deterministic baseline, and that a code-graph tool whose contribution is "finding imports" is rediscovering pipreqs. Its value must come from reachability, which is what F4 provides.

F10 — A measurement artifact in the published baseline

Experiment. While scoring the existing Qwen-7B patches, found that 35 of 98 contained only the filename rather than file contents.

Result. DI-Bench's output parser had consumed the fenced filename as the file body. **58 of 98 patches were parseable as stored; 95 of 98 were recoverable** from the raw model trajectories. Recovered scoring raised recall from 0.15 to 0.23.

Significance to the field. Reported benchmark numbers can be depressed by harness bugs rather than model capability. We also found and fixed two defects in DI-Bench's evaluation path: the fake-package check crashes on VCS-URL dependencies, and `act` prints "Job succeeded" for jobs that never started (a known upstream issue, nektos/act #5883), which an evaluator ignoring the exit code records as a pass.

Role in the argument. Supports the paper's thesis reflexively: measurement infrastructure needs auditing as much as the systems it measures. Belongs in Limitations or a short methods note, not as a headline.

F11 — Symbol-level reachability underperformed file-level [WRONG — do not publish]

What we reported. PyCG built call graphs for only 18 of 51 repositories and scored worse than the file-level signal (precision 0.47 vs 0.69).

Why it is wrong. The audit found that the `--package .` flag forces whole-program analysis including dependencies, which is exactly the configuration Jarvis (arXiv 2305.05949) documents as not scaling. A 2-file repository that times out at 240s *with* the flag completes in 0.2s without it. Drosos et al. ran PyCG across 1,302 projects at ecosystem scale. Worse, on the same 45 rows symbol-level **beats** file-level on F1 (0.545 vs 0.522); our script printed precision and recall separately and never computed F1.

Action required. Either re-run without the flag (or with Jarvis) and report F1, or demote to a one-line limitation: "we did not obtain symbol-level graphs at scale." Publishing it as a negative result would be a factual error a reviewer can reproduce.

F12 — An executable benchmark for in-situ dependency removal

Experiment. Derived repository-and-package pairs from gold-passing instances, and screened each by executing the deletion: an instance qualifies only if removing the declaration **provably fails CI**.

Result. 162 pairs over 47 repositories in three tiers. **Strong:** 62 pairs over 38 repositories, imported by at most two source files, test-reachable, deletion fails CI. **Medium:** 19 pairs, three to five files. **Weak:** 81 pairs whose deletion CI does not notice — excluded from scoring but shipped as a labelled catalogue of oracle blindness. Each instance carries a difficulty label: distinct symbols of the target that the project calls (1 to 22, median 1).

Significance to the field. Nothing in the literature does in-situ removal of a *used* dependency. PyTrim and DepClean remove *unused* declarations (the easy case). PyMigBench and MigrationBench migrate library A to library B. LibReuseBench and Commit0 are greenfield, with no existing behaviour to preserve. Mined removal datasets (Zenodo 12730734; He et al. 2021) record that removals happened but carry no executable environment, so they cannot score a candidate. **The execution-screened instance filter is what makes the oracle meaningful**, and is SWE-bench's FAIL_TO_PASS requirement applied to a manifest.

Role in the argument. The constructive deliverable. F2–F5 diagnose a broken oracle; F12 is the artifact that fixes it for one concrete task.

What would take this paper to a whole new level

Ranked by impact per unit of effort. The first three are what make a paper citable rather than merely publishable.

1. Release the audit as a reusable tool, not just a result

The move. Package the mutation audit as a one-command tool that any benchmark author can point at their own artifact: it deletes each ground-truth element, re-runs the oracle, and reports a blindness score with the phantom/blind-spot split.

Why it changes everything. Right now you have a finding about DI-Bench. A tool makes you *infrastructure*. STING and SWE-ABS became citable because they gave the field a method, not a number. Every future benchmark paper that wants to claim oracle quality would run your tool and cite you. This is the single

highest-leverage item, and the code already exists — it needs generalising from "manifest" to "any ground-truth element."

2. Audit three more benchmarks and report a cross-benchmark blindness table

The move. Apply the same mutation audit to SWE-bench Verified, a dependency-upgrade benchmark (DEPBENCH or TimeMachine-bench), and an environment-setup benchmark (EnvBench or Repo2Run). Report blindness per benchmark with the mechanism split.

Why. One benchmark is a case study; four is a *law*. It also directly tests the claim the audit flagged as unsupported ("we suspect many benchmarks would not survive"). If blindness varies systematically — say, lower where dependencies are pinned — that is a design principle the field can act on. Cost: compute only, no API spend.

3. Differential testing as the missing gate

The move. Implement gate 4 properly using PatchDiff-style differential testing (Wang, Pradel & Liu, ICSE 2026): generate inputs that distinguish the agent's replacement from the original library's behaviour, rather than relying on trivial-body heuristics.

Why. It converts your weakest gate into your strongest, and it would mechanically catch the two "genuine but degraded" removals we caught by hand — including the `wcswidth` case where a simplified Unicode implementation passes 490 tests because none cover East Asian width. This is the difference between "we propose a structural check" and "we have a working behavioural oracle."

4. Human reference solutions from mined removal commits

The move. Mine real dependency-removal commits from GitHub, attach them as reference solutions, and report the human-versus-agent gap on identical instances.

Why. It answers the question your F6/F7 results invite: is 3-of-28 bad, or is this task simply hard? A human baseline turns an absolute number into a calibrated one. It also upgrades the benchmark from "synthetic task" to "task developers demonstrably perform."

5. Scale the agent study to support rates, not just counts

The move. All 62 strong instances, three seeds, four to six models spanning capability tiers, both prompt conditions (neutral and prohibition-stated).

Why. Converts F6 and F7 from directional observations into statistics with confidence intervals, and definitively settles whether capability correlates with cheating. The prompt ablation is the cheapest high-value piece: if behaviour is identical with and without explicit prohibitions, the gates are catching something models do not do anyway, which is itself publishable.

6. Extend beyond Python

The move. Repeat the mutation audit on DI-Bench's C#, Rust and JavaScript subsets.

Why. The mechanisms (transitive installation, uneven coverage) are not Python-specific but their magnitudes should vary sharply with ecosystem strictness. Rust's Cargo resolves dependencies far more strictly than pip, so if phantoms nearly vanish there, you have shown that *ecosystem design* determines oracle quality — a genuinely novel and quotable claim.

7. The temporal study

The move. Re-run the gold patches at intervals and model the decay curve; or reconstruct historical snapshots to measure decay retrospectively.

Why. F1 is a single point. A curve tells the field how long a benchmark stays valid, which is actionable for everyone who builds one. "Executable benchmarks lose roughly X% of validity per year" would be cited by every benchmark paper written afterwards.